

Road Network Planning with Track Map

Road Network Planning with Track Map

1. Course Content
2. Track Lane Network Analysis
3. Start Agent
4. Build Track Grid Map and Pose Map
5. Annotate Road Network
6. Start Road Network Navigation
7. Test Track Road Network Navigation Case
 - 7.1 Point-to-Point Navigation Between Road Networks
 - 7.2 Arbitrary Pose Road Network Planning Navigation

1. Course Content

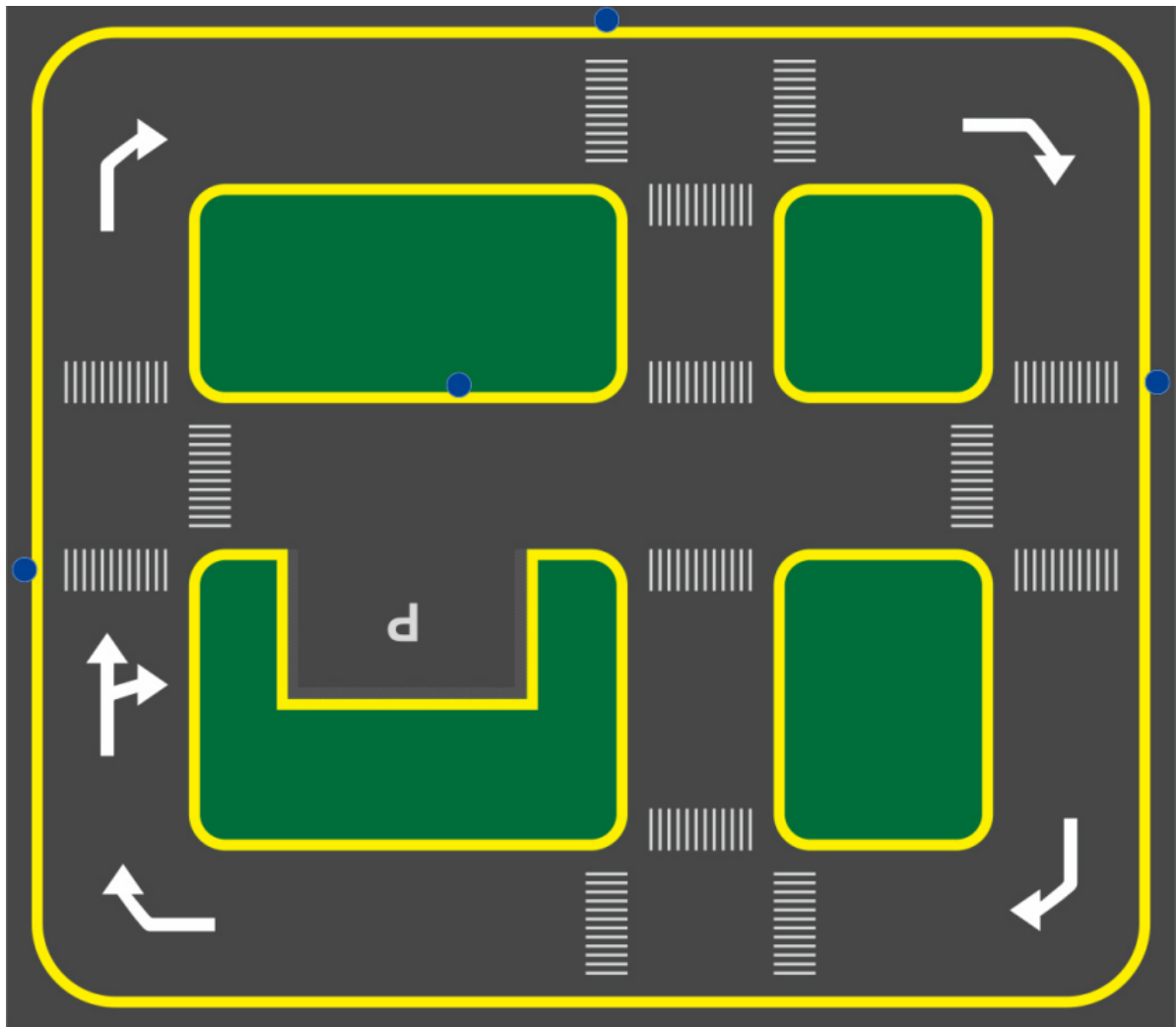
[!IMPORTANT]

- This course is an extended course for the track map and requires the use of the accompanying track map to proceed with the course case study
- **This case study is not applicable to Raspberry Pi 5 users!!**

1. Use road network planning navigation, combined with functional track map to build urban road networks, allowing the vehicle to navigate and drive in the track while following lane traffic rules and planning paths that conform to the road network
2. Subsequent courses will include AI large model combined with track autonomous driving and road network planning navigation case studies. This section explains how to build road network planning navigation in the track

2. Track Lane Network Analysis

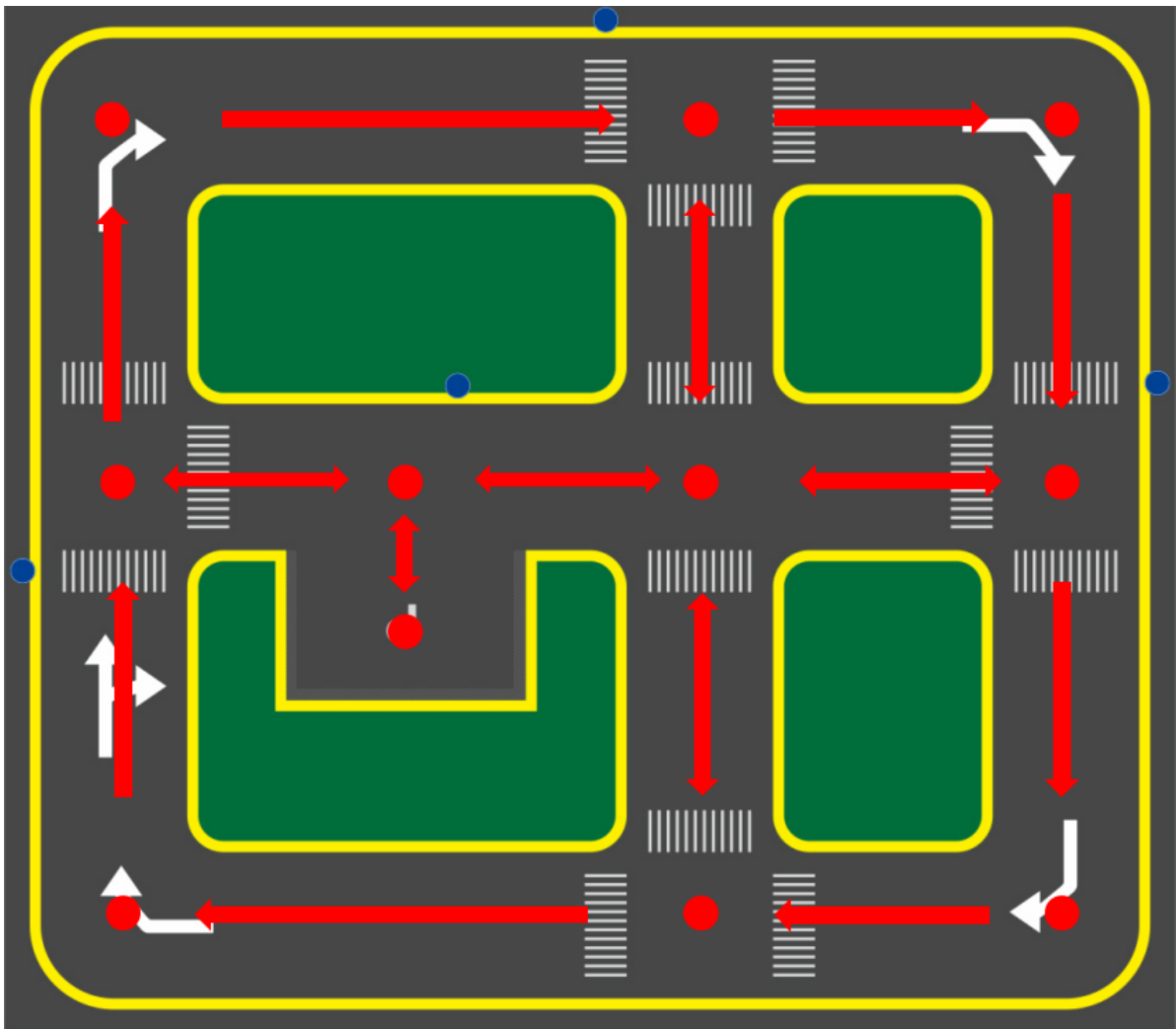
- Here we can find that the outermost lane is a **one-way lane**, which can only travel clockwise in one direction, while the middle lane is a two-way lane. Therefore, we can use all corners and intersections as road points to connect edges and build the entire lane's topological network



- The lane traffic rules network is shown in the following figure

[!TIP]

The figure below shows an example lane traffic network. Users can also add more road points or modify the lane network direction according to their actual needs



3. Start Agent

- Use the following command to start the microROS agent and connect to the chassis

[!WARNING]

If you use the `start_agent` command to start the microROS agent, you need to close the previous agent first. Note that this is different from the previous microROS agent startup command

```
start_kilted_agent
```

```
jetson@yahboom: ~
jetson@yahboom: ~ 80x24
ubsubscriber created | client_key: 0x14F22590, subscriber_id: 0x000(4), partici
pant_id: 0x000(1)
[1763535321.445792] info | ProxyClient.cpp | create_datareader | d
atareader created | client_key: 0x14F22590, datareader_id: 0x000(6), subscri
ber_id: 0x000(4)
[1763535321.449556] info | ProxyClient.cpp | create_topic | t
opic created | client_key: 0x14F22590, topic_id: 0x006(2), participant_
id: 0x000(1)
[1763535321.452876] info | ProxyClient.cpp | create_subscriber | s
ubscriber created | client_key: 0x14F22590, subscriber_id: 0x001(4), partici
pant_id: 0x000(1)
[1763535321.457712] info | ProxyClient.cpp | create_datareader | d
atareader created | client_key: 0x14F22590, datareader_id: 0x001(6), subscri
ber_id: 0x001(4)
[1763535321.461959] info | ProxyClient.cpp | create_topic | t
opic created | client_key: 0x14F22590, topic_id: 0x007(2), participant_
id: 0x000(1)
[1763535321.466141] info | ProxyClient.cpp | create_subscriber | s
ubscriber created | client_key: 0x14F22590, subscriber_id: 0x002(4), partici
pant_id: 0x000(1)
[1763535321.473112] info | ProxyClient.cpp | create_datareader | d
atareader created | client_key: 0x14F22590, datareader_id: 0x002(6), subscri
ber_id: 0x002(4)
```

- Modify ROS communication middleware DDS

[!WARNING]

Open ~/.bashrc, uncomment this line, and change it to the DDS communication middleware as shown in the figure (only the road network planning navigation chapter needs to use cyclondds, **please restore the environment variables after completing the road network chapter!!**)

```
vim ~/.bashrc
```

- RDK mainboard

```
131 #ROSMaster-M3 Code space environment variables
132 export PYTHONPATH="/home/sunrise/yahboomM3_ws/.venv/lib/python3.10/site-packages:$PYTHONPATH"
133 export ROS_DOMAIN_ID=16
134 export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp
135 #unset RMW_IMPLEMENTATION
136 export ROBOT_TYPE="ROSMaster-M3"
137 export MULTI_ROBOT="" #robot1 or robot2
138 #=====雷达和相机型号=====
139 #=====Radar and camera models=====
140 #=====
141 #单雷达: single          双雷达: dual
142 #Single radar: single    Dual radar: dual
143 export LASER_TYPE=single
144 #export LASER_TYPE=dual
```

- ORIN mainboard

```
133 export ROS_DOMAIN_ID=30
134 export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp
135 export ROBOT_TYPE="ROSMaster-M3"
136 export MULTI_ROBOT=""
```

```
:wq #Save and exit  
source ~/.bashrc #Update environment
```

4. Build Track Grid Map and Pose Map

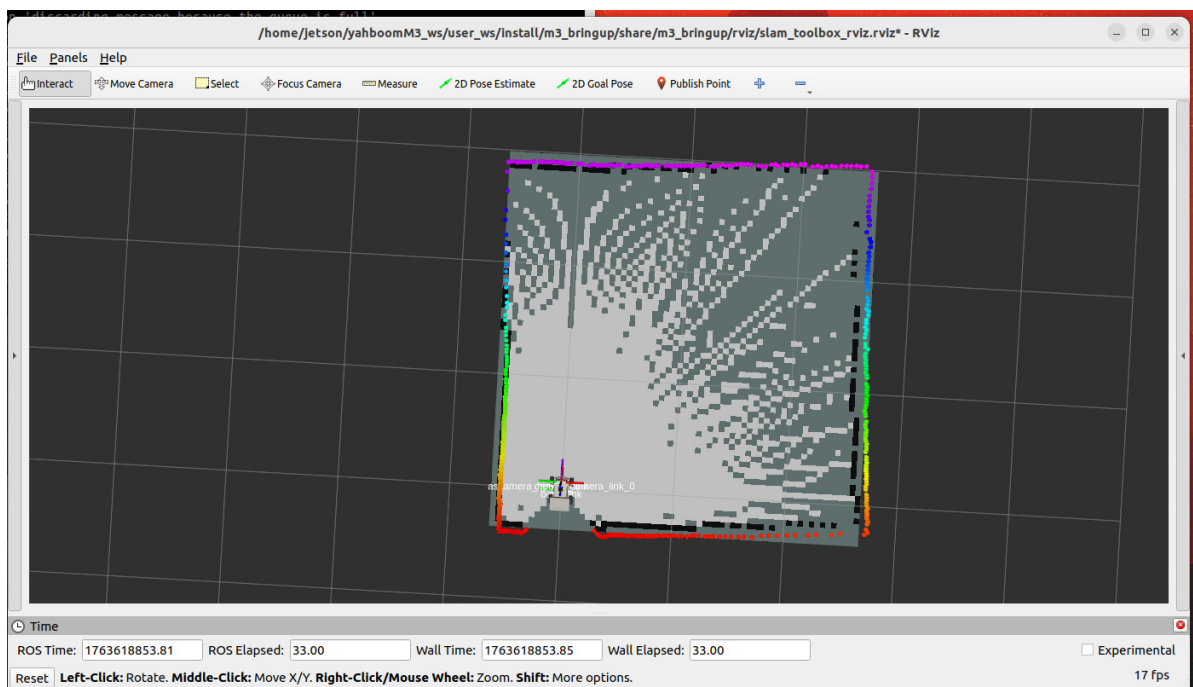
- Enter command in terminal

(ORIN mainboard)

```
slam
```

(RDK mainboard) Run step by step, visualization is started on the accompanying virtual machine
Rosmaster-M3-Ubuntu22.04 ,

```
#Virtual machine terminal  
rviz2 -d  
/home/yahboom/yahboomM3_ws/user_ws/src/m3_bringup/rviz/slam_toolbox_rviz.rviz  
#Host terminal  
ros2 launch m3_bringup slam_toolbox.launch.py gui:=False
```



- Then start the keyboard control node (you can also use a controller for control)

```
keyboard_control
```

- Control the vehicle to move for mapping

```
u   i   o
j   k   l
m   ,   .

For Holonomic mode (strafing), hold down the shift key:
-----
U   I   O
J   K   L
M   <   >

t : up (+z)
b : down (-z)

anything else : stop

q/z : increase/decrease max speeds by 10%
w/x : increase/decrease only linear speed by 10%
e/c : increase/decrease only angular speed by 10%

CTRL-C to quit

currently:      speed 0.5      turn 1.0
█
```

- Save grid map

[!NOTE]

Here the naming is based on `citymap_4` as an example, where `citymap_4` is the map name and can be set by yourself

```
ros2 launch m3_bringup save_map.launch.py map_name:=citymap_4
```

```
jetson@yahboom: ~ 82x22
[map_saver_cli-1] map_saver lifecycle node launched.
[map_saver_cli-1] Waiting on external lifecycle transitions to activate
[map_saver_cli-1] See https://design.ros2.org/articles/node_lifecycle.html for more information.
[map_saver_cli-1] [INFO] [1763532975.329685271] [map_saver]: Creating
[map_saver_cli-1] [INFO] [1763532975.330069728] [map_saver]: Configuring
[map_saver_cli-1] [INFO] [1763532975.341867665] [map_saver]: Saving map from 'map'
[map_saver_cli-1] topic to '/home/jetson/map/map' file
[map_saver_cli-1] [WARN] [1763532975.359932772] [map_io]: Image format unspecified
[map_saver_cli-1] . Setting it to: pgm
[map_saver_cli-1] [INFO] [1763532975.361316136] [map_io]: Received a 224 X 598 map
[map_saver_cli-1] [INFO] [1763532975.439369065] [map_io]: Writing map occupancy data to /home/jetson/map/map.pgm
[map_saver_cli-1] [INFO] [1763532975.443013992] [map_io]: Writing map metadata to /home/jetson/map/map.yaml
[map_saver_cli-1] [INFO] [1763532975.444847063] [map_io]: Map saved
[map_saver_cli-1] [INFO] [1763532975.446514786] [map_saver]: Map saved successfully
[map_saver_cli-1] [INFO] [1763532975.449017438] [map_saver]: Destroying
[INFO] [map_saver_cli-1]: process has finished cleanly [pid 50218]
jetson@yahboom:~$
```

- Save slam_toolbox pose map

```
ros2 launch m3_bringup save_map.launch.py map_name:=citymap_4
map_type:=posegraph
```

```
[INFO] [launch]: All log files can be found below /home/jetson/.ros/log/2025-11-19-14-19-41-297598-yahboom-66778
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [ros2-1]: process started with pid [67023]
[ros2-1] requester: making request: slam_toolbox.srv.SerializePoseGraph_Request(filename='/home/jetson/map/map')
[ros2-1]
[ros2-1] response:
[ros2-1] slam_toolbox.srv.SerializePoseGraph_Response(result=0)
[ros2-1]
[INFO] [ros2-1]: process has finished cleanly [pid 67023]
jetson@yahboom:~$
```

5. Annotate Road Network

- Start navigation node on host machine

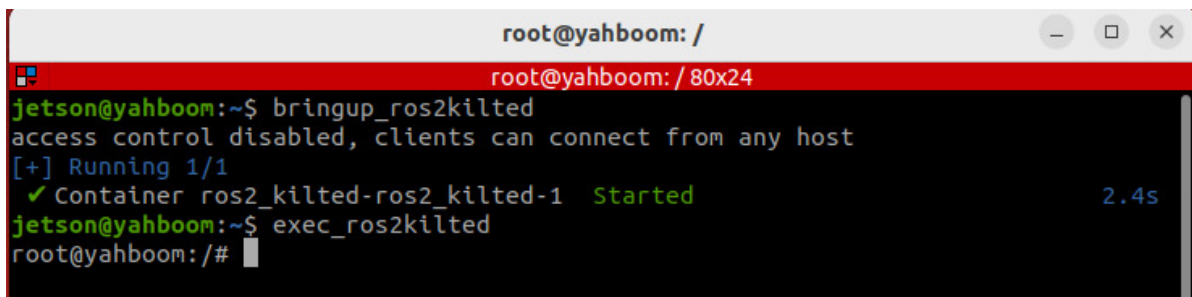
```
ros2 launch m3_bringup nav2.launch.py map:=citymap_4.yaml pose_map:=citymap_4
localization_mode:=slam_toolbox gui:=False
```

- Start ros2_kilted container (skip for RDK mainboard)

```
bringup_ros2kilted
```

- Enter ros2_kilted container terminal (skip for RDK mainboard)

```
exec_ros2kilted
```



```
root@yahboom: /
jetson@yahboom:~$ bringup_ros2kilted
access control disabled, clients can connect from any host
[+] Running 1/1
  ✓ Container ros2_kilted-ros2_kilted-1 Started 2.4s
jetson@yahboom:~$ exec_ros2kilted
root@yahboom:/#
```

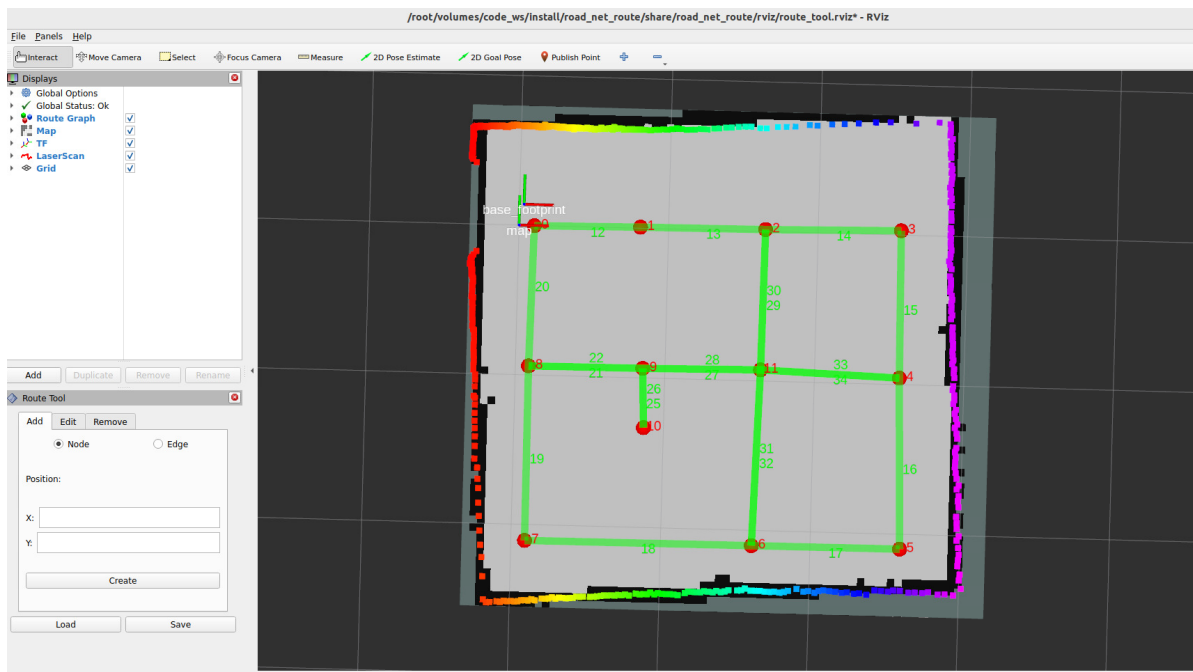
- Start road network annotation node, start in ros2_kilted container terminal (skip for RDK mainboard)

```
ros2 launch road_net_route route_tool.launch.py yaml_filename:=citymap_4.yaml
```

- (RDK mainboard) Only need to start rviz, visualization is started on the accompanying virtual machine `Rosmaster-RoadNet-Ubuntu24.04`,

```
#Virtual machine terminal
rviz2 -d /home/yahboom/code_ws/src/road_net_route/rviz/route_tool.rviz
```

- For specific operations of calibrating and saving the road network, refer to the `03-Road Network Annotation` section course in this chapter's tutorial
- The following is an example calibrated lane network



6. Start Road Network Navigation

- Open a terminal on the host machine to start the vehicle's radar fusion node and odometer fusion

```
ros2 launch m3_bringup car_base.launch.py
```

- Start ros2_kilted container

```
bringup_ros2kilted
```

- Open a container terminal

```
exec_ros2kilted
```

- (RDK mainboard) Visualization is started on the accompanying virtual machine `Rosmaster-RoadNet-Ubuntu24.04`,

```
rviz2 -d /home/yahboom/code_ws/src/road_net_route/rviz/nav2_view.rviz
```

- Start road network planning navigation node

```
ros2 launch road_net_route roadnet_nav2.launch.py map:=citymap_4.yaml  
pose_map:=citymap_4 roadnet_file:=/root/map/citymap_4.geojson
```

[!TIP]

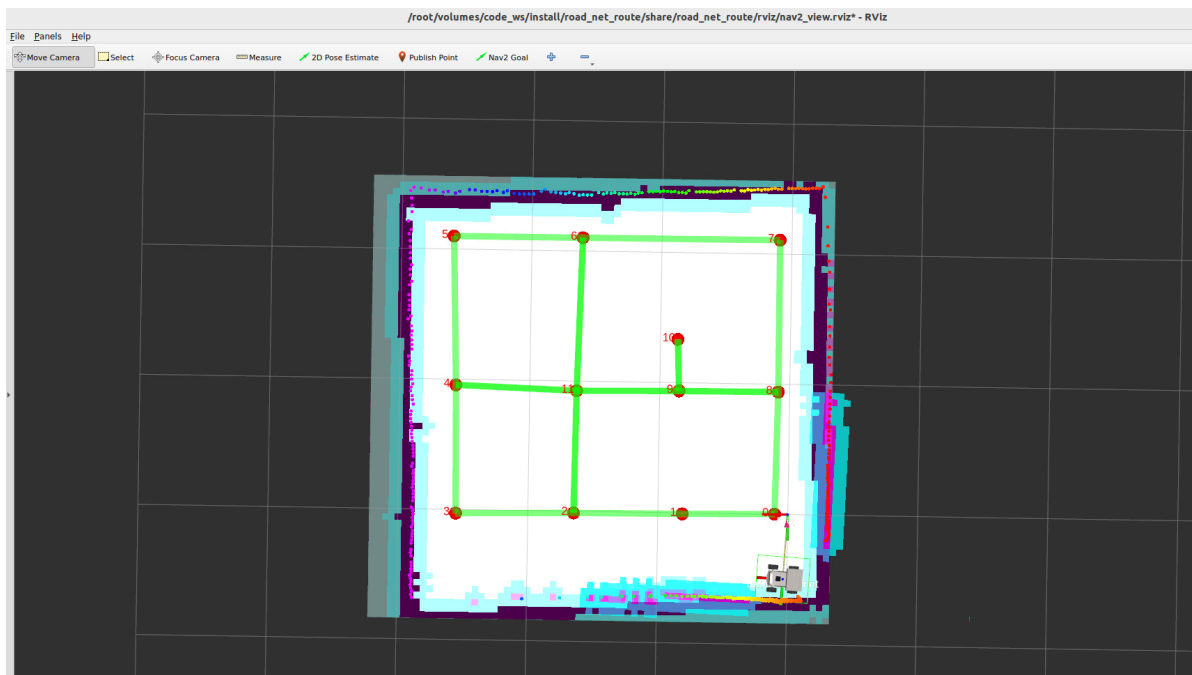
Parameter description

Note: The map name passed in here should be changed to your actual saved map name. If it is the default name, you don't need to pass map parameters

map Grid map file name to be loaded, default value is `map.yaml`

pose_map Pose map file to be loaded. When navigation global positioning uses slam_toolbox pure positioning, the pose map must be passed in, default value is `map`

roadnet_file Road network description file, default value is: `map.geojson`

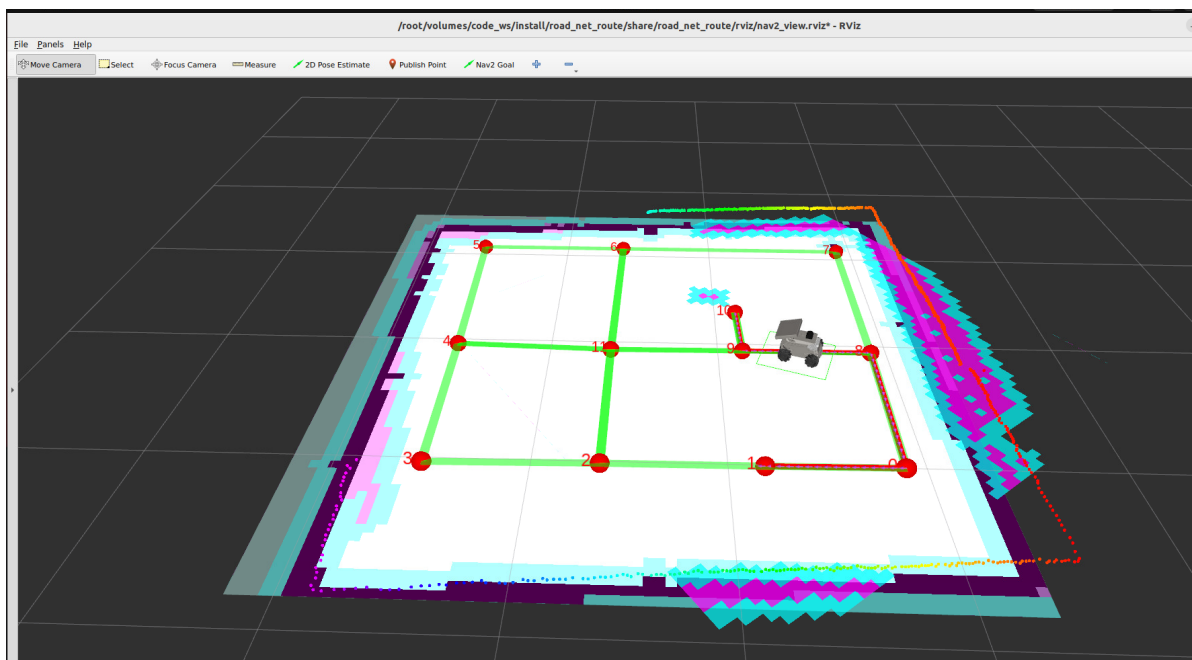


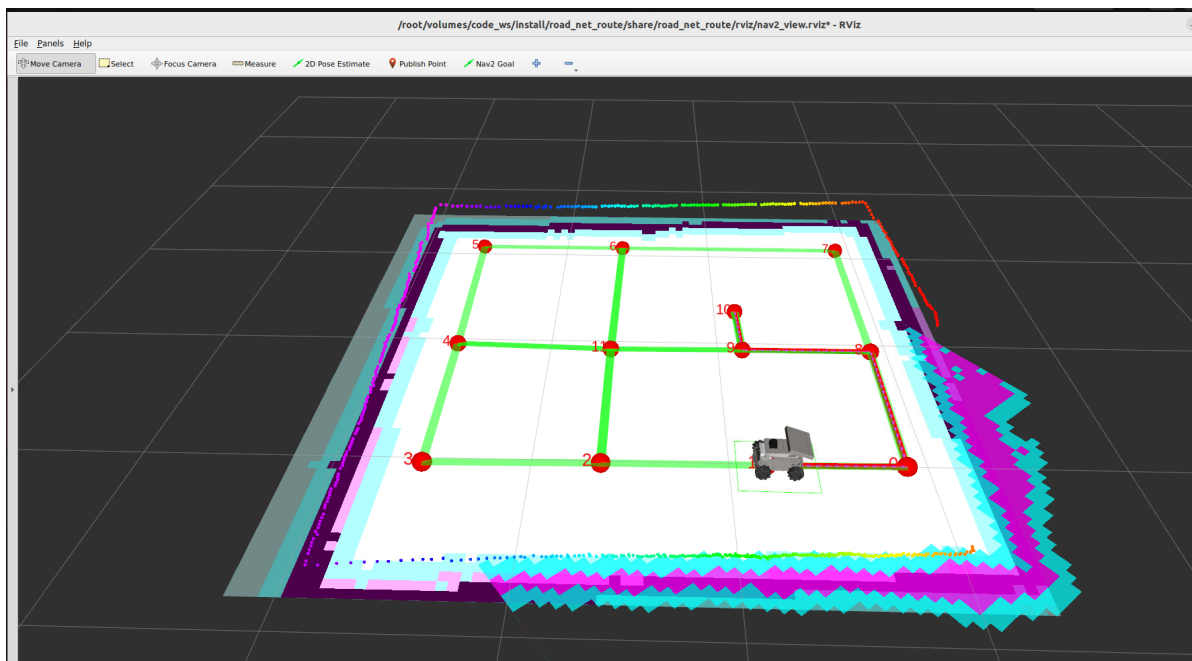
7. Test Track Road Network Navigation Case

7.1 Point-to-Point Navigation Between Road Networks

- Here we demonstrate navigating from road network point 10 to road network point 1 as an example. Publish the topic in the terminal

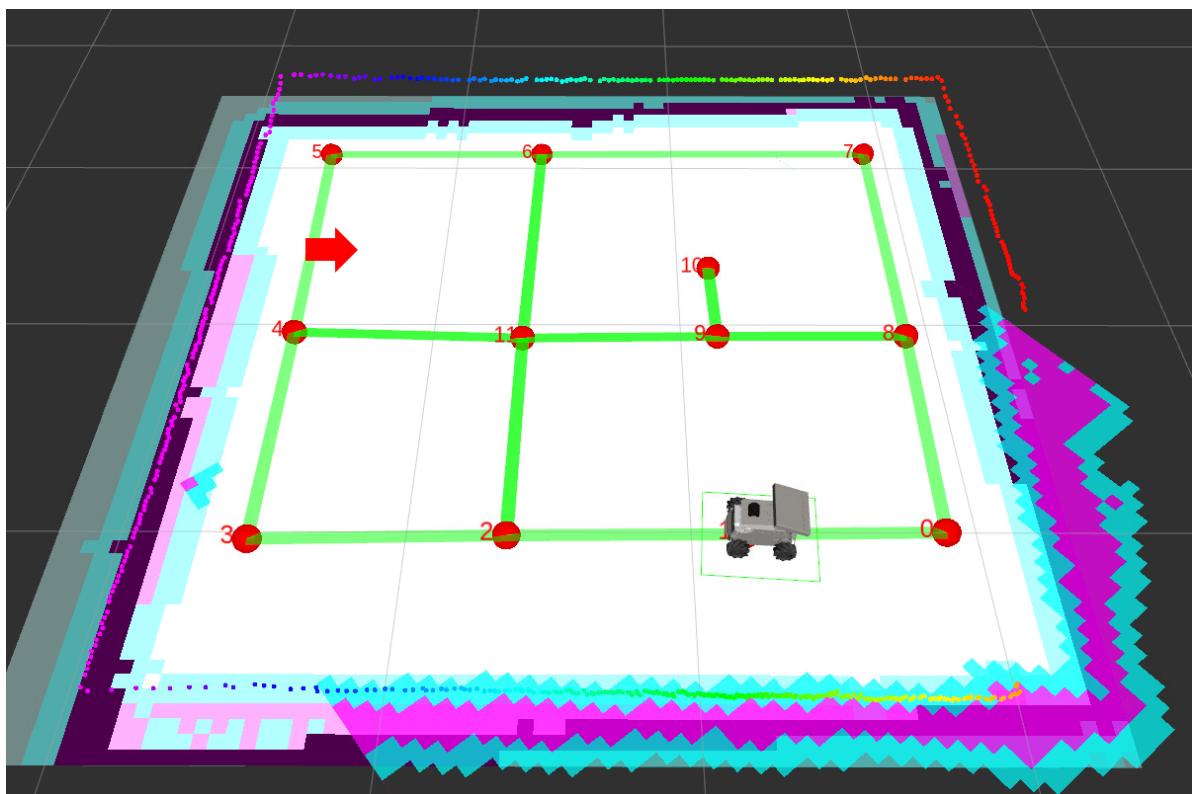
```
ros2 topic pub -1 /road_net_nav_id std_msgs/msg/Int16MultiArray "{data: [10, 1]}"
```



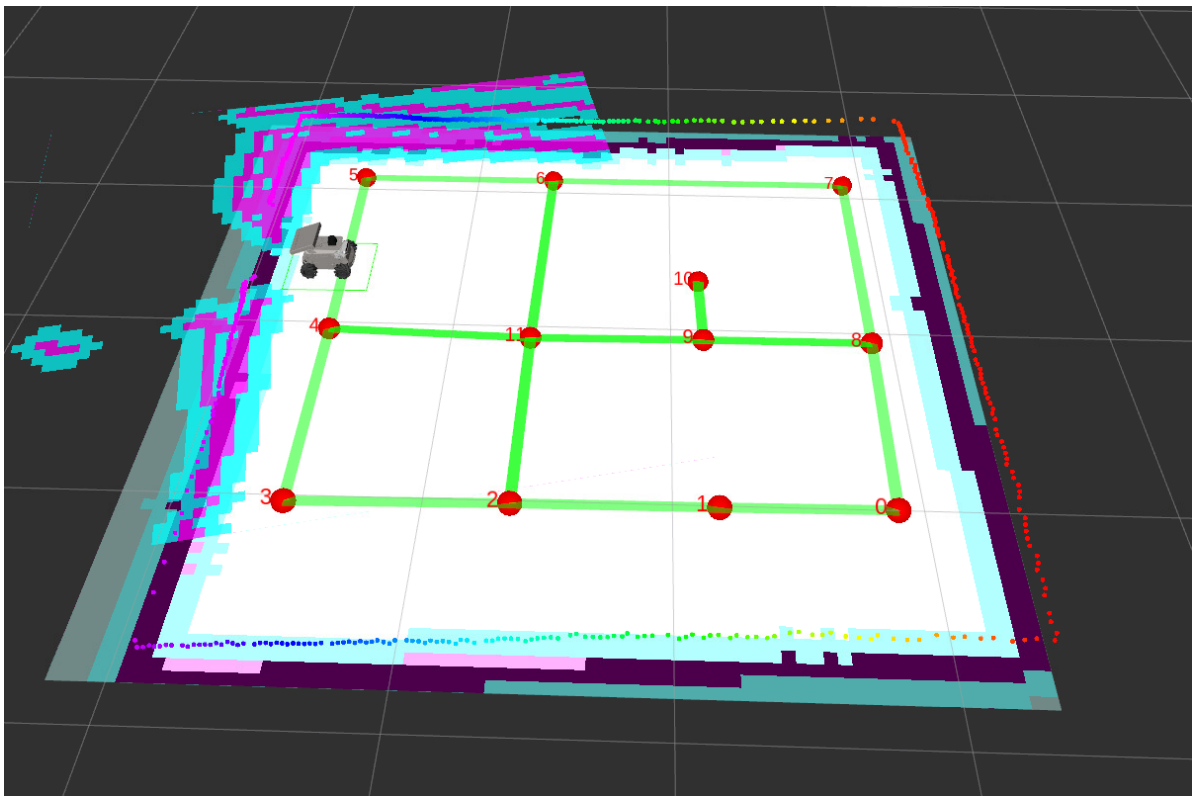


7.2 Arbitrary Pose Road Network Planning Navigation

- Here we demonstrate with the arrow direction in the figure as an example



- First control the vehicle to move to this pose

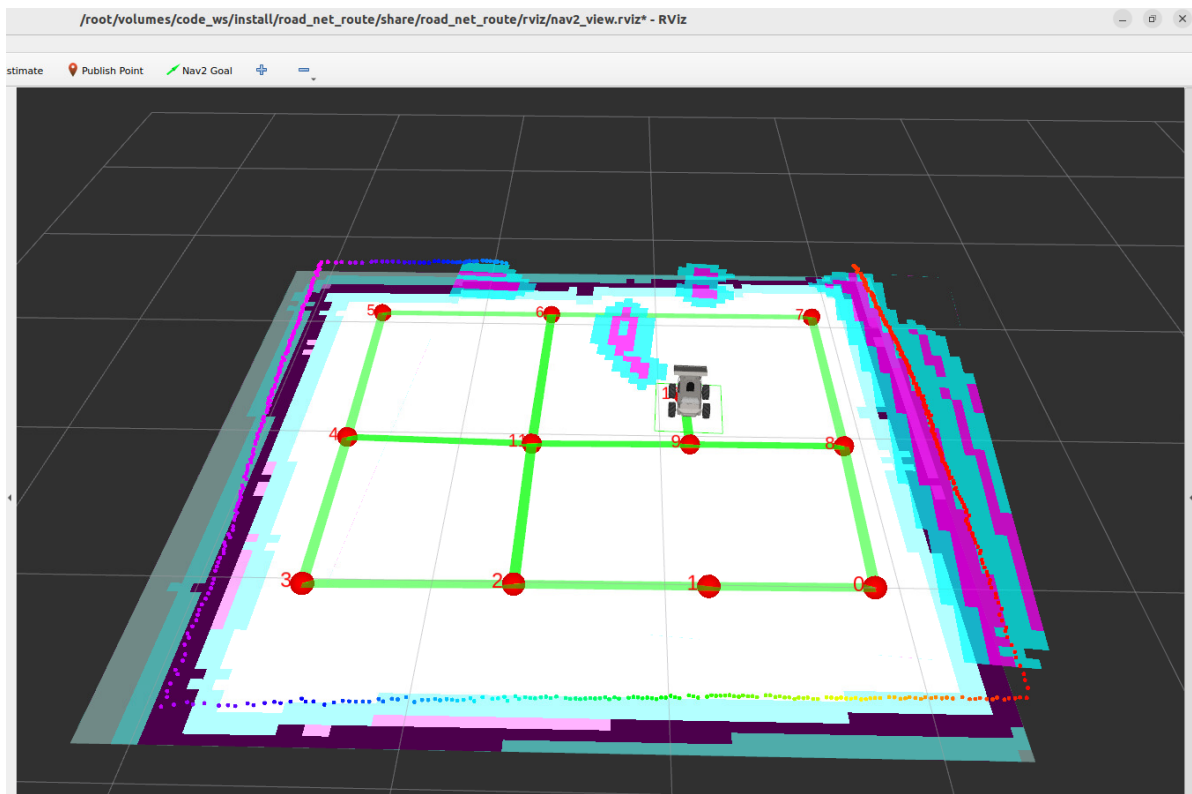


- Use the following command to view the current robot's coordinates in the map

```
ros2 run tf2_ros tf2_echo map base_footprint
```

```
jetson@yahboom: ~ 68x18
- Rotation: in RPY (degree) [0.000, -0.000, 179.949]
- Matrix:
-1.000 -0.001  0.000  2.648
 0.001 -1.000  0.000 -1.388
 0.000  0.000  1.000  0.000
 0.000  0.000  0.000  1.000
At time 1763622288.978000000
- Translation: [2.648, -1.388, 0.000]
- Rotation: in Quaternion (xyzw) [0.000, 0.000, 1.000, 0.000]
- Rotation: in RPY (radian) [0.000, -0.000, 3.141]
- Rotation: in RPY (degree) [0.000, -0.000, 179.985]
- Matrix:
-1.000 -0.000  0.000  2.648
 0.000 -1.000  0.000 -1.388
 0.000  0.000  1.000  0.000
 0.000  0.000  0.000  1.000
^C[INFO] [1763622290.212653804] [rclcpp]: signal_handler(signum=2)
jetson@yahboom: ~$
```

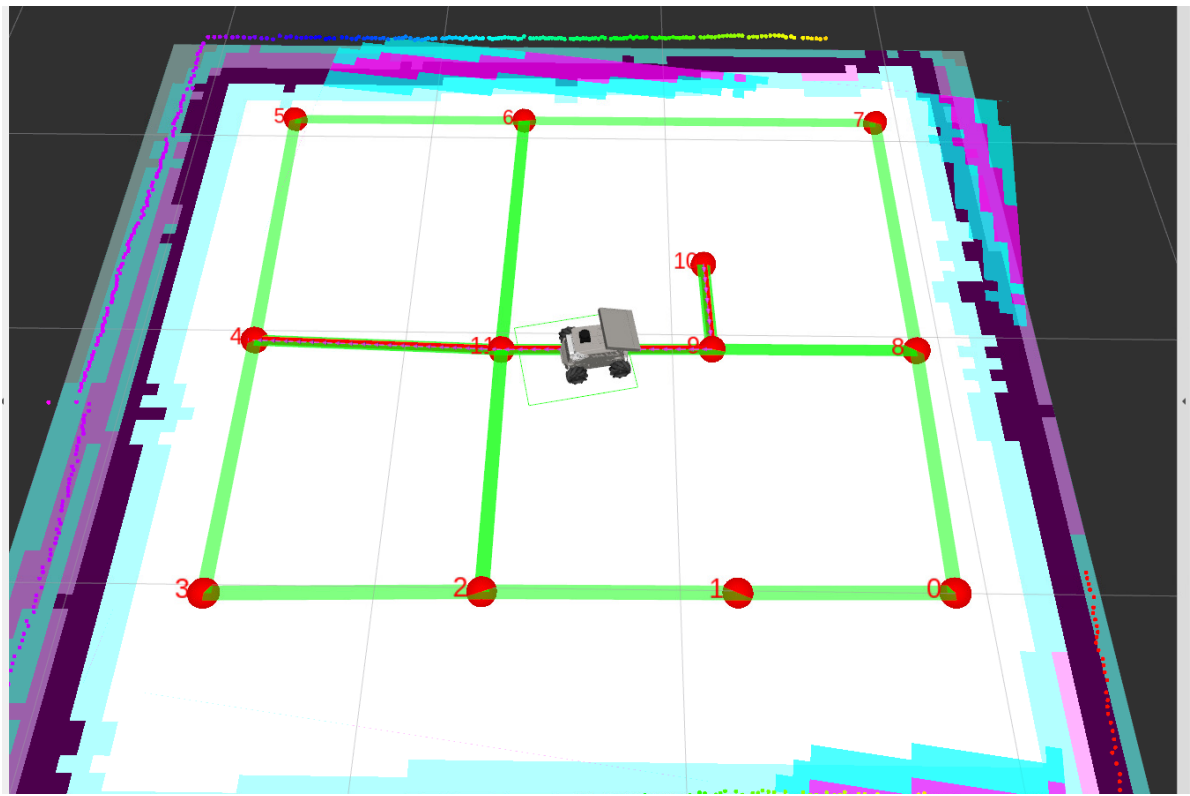
- Then let the vehicle return to road network point 10 (here we use road network point 10 as an example, it can be at any position in the map or in the road network)



- Publish target pose topic in terminal

```
ros2 topic pub -1 /road_net_nav geometry_msgs/PoseStamped "{
  header: {frame_id: 'map'},
  pose: {
    position: {x: 2.648, y: -1.388, z: 0.0},
    orientation: {x: 0.0, y: 0.0, z: 1.000, w: 0.000}
  }
}"
```

- The vehicle will first reach the road network point nearest to the target pose along the road network



- Then adjust to the target pose

